

Order reduction by co-contraction

A complete theory guide for `order-reduction-sim`

Working document for the Royal Society URF proposal

Repository: `order-reduction-sim/report/theory.tex`

September 2026

How to read this document (interactive)

This PDF is **hyperlinked**. Click any blue section name in the [Notation table \(§2\)](#) or in the [table of contents](#) to jump there.

Suggested path for a first read:

1. §1 — the cup-and-arm story in one page.
2. §3–§4 — where $\rho(\xi)$ comes from.
3. §7 — the bias law (the main finding).
4. §9 — Learner 2 and what it discovered.
5. §11 — what we conclude and what we do *not* claim.

Green boxes are confirmed results. Red boxes are refuted claims or limits. Yellow boxes are everyday examples.

Contents

1	The story in one page	3
2	Complete notation table	3
3	Part I — The second-order mass–spring–damper	4
3.1	The physical equation	4
3.2	Laplace transform and transfer function	4
3.3	Natural frequency and damping ratio	4
3.4	The two poles	5
4	Part II — Time-scale separation and $\rho(\xi)$	5
4.1	Separation ratio	5
4.2	Singular perturbation: the reduced first-order plant	6
5	Part III — The third-order object in the simulation	6
5.1	Object transfer function	6
5.2	What co-contraction changes	6
5.3	Discrete-time simulation	7
5.4	Movement and exploration	7
6	Part IV — Trial failure (surprise)	7
7	Part V — The bias law (central finding)	7
7.1	Low-frequency expansion	7
7.2	Effect of co-contraction	8

8	Part VI — Learner 1 (gradient descent, order unlocked by hand)	9
8.1	What it believes	9
8.2	Prediction	9
8.3	Learning rule	9
8.4	Conditions S1–S5	9
8.5	Results (20 seeds, 80 trials)	9
9	Part VII — Learner 2 (two-timescale EKF + ARD)	10
9.1	State vector	10
9.2	Blend dynamics (why not just set $\tau = 0$?)	10
9.3	Effective order	11
9.4	Extended Kalman Filter (EKF) — full plain-language guide	11
9.5	ARD — Automatic Relevance Determination	11
9.6	Two timescales	12
9.7	Conditions K1–K5	12
9.8	Results (20 seeds, 60 trials)	12
10	Part VIII — Scoring metrics (every formula)	13
11	Part IX — Conclusions and discussion	13
11.1	What we conclude	13
11.2	What we do <i>not</i> conclude	13
11.3	Recommended claim for the URF four-pager	13
11.4	Open next steps (not yet simulated)	14
12	Appendix — Code map	14

1 The story in one page

You hold a cup of coffee and learn to move it without spilling. The cup has its own physics (weight, wobble, slosh). Your arm has physics too (stiffness, damping). Together they form one **coupled system**.

Thrishanth Nanayakkara’s idea, developed after the July 2026 meeting, is:

When the coupled system is **high order** (many ways to wobble), the brain may **stiffen the arm** first. Stiffening pushes the fast wobbles so far into the past that, during one reach, you mostly feel the **slow, dominant** response. You learn that slow part safely. Then you **relax** and learn the fast parts on top.

The simulation in **order-reduction-sim** tests this idea with two different computer learners. Both use the same cup and the same five tuning schedules; they differ in *how* they learn.

Lay example: Cup of coffee

Soft arm, fast reach: the coffee sloshes, the cup wobbles, you feel many things at once. If your mental model is wrong, the movement fails (spill).

Stiff arm, slow reach: you squeeze hard. Slosh dies in milliseconds. During your 0.8 s reach you mostly feel “how heavy/sluggish is this cup?” — one main lag. Safer, simpler, but you cannot yet learn the slosh.

Then relax: once you know the main lag, you soften the arm and move faster. The slosh reappears in what you feel. Now you can learn it — but only if your coarse model is already good enough that the reach does not fail.

2 Complete notation table

Every symbol used in this document and in the code is listed here. Click any **Symbol** entry’s description to jump to the section where it is defined in context.

Symbol	Units	Meaning
$x(t)$	m	Position of the coupled hand–object along one axis.
$F(t)$	N	Force applied (your muscle command).
M	kg	Effective inertia (mass of arm + object).
B	N s/m	Damping (friction + muscle viscosity). <i>You increase this by co-contracting.</i>
K	N/m	Stiffness (muscle tone + object stiffness). <i>You increase this too.</i>
ξ	—	Damping ratio (co-contraction level in the simulation).
ω_n	rad/s	Natural frequency, $\sqrt{K/M}$.
p_d	1/s	Dominant (slow) pole — how sluggish the system feels.
p_f	1/s	Fast (transient) pole — how quickly wobbles die.
$\rho(\xi)$	—	Separation ratio $ p_f / p_d $ when $\xi > 1$. Grows rapidly with stiffness.
s	1/s	Laplace variable (frequency domain).
$G(s)$	—	Transfer function: output per unit input in frequency domain.
τ_d, τ_1, τ_2	s	Time constants of the three-lag object model (slow, medium, fast).
$\hat{\tau}_d, \hat{\tau}_1, \hat{\tau}_2$	s	Learner’s estimates of those time constants.
$u(t)$	—	Command signal sent to the limb (reference reach + exploration).
$y(t)$	—	Felt response (position or force trace).
$\hat{y}(t)$	—	Learner’s prediction of what it should have felt.
ξ	—	Co-contraction; same symbol as damping ratio in the coupled model.
Δt or Δt	s	Sample interval (0.01 s in code).
t_{move}	s	Duration of one reach segment (controls movement speed).

Symbol	Units	Meaning
σ	—	Sensor / output noise standard deviation.
surprise	—	$\max_t y(t) - \hat{y}(t) $; largest prediction mismatch in a trial.
S_BASE	—	Failure threshold scale (0.15 in code).
n_{unlocked}	—	How many time constants Learner 1 is allowed to fit (1, 2, or 3).
w_1, w_2	—	Participation weights (Learner 2): how active each transient mode is (0–1).
n_{eff}	—	Effective order = $1 + w_1 + w_2$ (Learner 2).
q	—	State vector in log-parameters + weights (Learner 2).
P	—	Covariance matrix: how uncertain the learner is.
Q	—	Process noise covariance: how much beliefs drift per trial.
R	—	Measurement noise variance in the filter update.
H or J	—	Jacobian: sensitivity of prediction to each parameter.
α_i	—	ARD precision (penalty) on weight w_i .
$\kappa(\Phi)$	—	Condition number of the identification matrix (hard vs easy to learn).

3 Part I — The second-order mass–spring–damper

3.1 The physical equation

Along one movement axis, the coupled arm and object are modelled as a **mass** pulled by a **spring** and slowed by a **dashpot** (damper):

$$M\ddot{x} + B\dot{x} + Kx = F(t) \quad (1)$$

- $M\ddot{x}$ — inertia resists *acceleration* (“hard to get moving”).
- $B\dot{x}$ — damping resists *velocity* (“motion is slowed, wobbles die”).
- Kx — stiffness pulls back toward rest (“rubber band”).
- $F(t)$ — your muscle force.

Lay example: Mass–spring–damper

Push a shopping trolley. M is the trolley mass. K is how stiffly you hold the handle (and any elastic in the object). B is friction in the wheels and how much you resist the handle moving. Co-contraction in the arm increases both B and K — you grip tighter and resist motion more.

3.2 Laplace transform and transfer function

Take the Laplace transform of (1) (zero initial conditions):

$$Ms^2X(s) + BsX(s) + KX(s) = F(s)$$

Solve for the ratio $X(s)/F(s)$:

$$G(s) = \frac{X(s)}{F(s)} = \frac{1}{Ms^2 + Bs + K} \quad (2)$$

This $G(s)$ is the **transfer function**: it tells you how each frequency in F is amplified or delayed in x .

3.3 Natural frequency and damping ratio

Define:

$$\omega_n = \sqrt{\frac{K}{M}} \quad (\text{natural frequency, rad/s}) \quad (3)$$

$$\xi = \frac{B}{2\sqrt{MK}} \quad (\text{damping ratio, dimensionless}) \quad (4)$$

Lay example: What ξ means

- $\xi < 1$: underdamped — oscillates before settling (bouncy).
- $\xi = 1$: critically damped — returns to rest as fast as possible without overshoot.
- $\xi > 1$: overdamped — no oscillation; two real exponential decays stacked.

Co-contraction in our simulation is mapped to **raising ξ past 1**.

3.4 The two poles

The denominator $Ms^2 + Bs + K = 0$ has roots (poles):

$$p_{\pm} = -\omega_n(\xi \pm \sqrt{\xi^2 - 1}) \quad (5)$$

For $\xi > 1$ both roots are real and negative. Write:

$$p_d \approx -\frac{K}{B} \quad (\text{dominant, **slow** pole}) \quad (6)$$

$$p_f \approx -\frac{B}{M} \quad (\text{fast, **transient** pole}) \quad (7)$$

Lay example: Two poles = two time scales

After a push, the response is roughly a sum of two decays:

$$x(t) \approx A_d e^{p_d t} + A_f e^{p_f t}$$

Because $|p_f| \gg |p_d|$, the fast term $e^{p_f t}$ dies quickly; the slow term lingers. That slow term is what you “feel” during a normal reach.

4 Part II — Time-scale separation and $\rho(\xi)$

4.1 Separation ratio

Define the **separation ratio**:

$$\rho(\xi) = \frac{|p_f|}{|p_d|} \quad (8)$$

For $\xi > 1$, substituting (6)–(7) and using (3)–(4), one obtains the **closed form** used everywhere in the code (`plant.rho`):

$$\rho(\xi) = (\xi + \sqrt{\xi^2 - 1})^2 \quad \text{for } \xi > 1, \quad \rho(\xi) = 1 \quad \text{for } \xi \leq 1 \quad (9)$$

Lay example: Numbers for $\rho(\xi)$

ξ	$\rho(\xi)$	Plain English
0.7	1.0	Soft: modes not separated ($\xi < 1$ in strict MSD, but simulation uses $\rho = 1$).
1.2	3.5	Fast mode 3.5× faster than slow.
2.0	13.9	Strong separation.
3.0	34.0	Very stiff: transients finish in $\sim 1/34$ of the slow time.
4.0	62.0	Extreme stiffening.

4.2 Singular perturbation: the reduced first-order plant

When $\rho \gg 1$, the fast mode settles in a time $\tau_f = 1/|p_f|$ that is negligible compared with the movement duration T . Standard **singular perturbation** replaces the fast mode by its quasi-steady state and yields a **first-order** reduced model:

$$\tau_d \dot{x} + x = \frac{1}{K} F, \quad \tau_d = \frac{B}{K} \quad (10)$$

Lay example: Reduced model

While very stiff, the coupled system *behaves during one reach* like a single lag τ_d : push $\Rightarrow$ response ramps up with time constant τ_d . The fast wobble happened already — you did not see it. The cup did not change; your arm **filtered** the experience.

5 Part III — The third-order object in the simulation

The code does not simulate (1) and the object separately on every time step. It uses a **phenomenological stand-in** that captures the *outcome* of coupling: time-scale separation on a fixed object.

5.1 Object transfer function

The **object alone** (the cup) is modelled as three first-order lags in series:

$$G(s) = \frac{1}{(1 + \tau_d s)(1 + \tau_1 s)(1 + \tau_2 s)} \quad (11)$$

True values in the repository (TAU_TRUE):

$$\tau_d = 1.0 \text{ s}, \quad \tau_1 = 0.1 \text{ s}, \quad \tau_2 = 0.05 \text{ s}$$

Poles at $s = -1, -10, -20$ rad/s.

Lay example: Three lags in a row

Think of three filters in series:

1. Slow filter (τ_d): main heaviness.
2. Medium filter (τ_1): first wobble.
3. Fast filter (τ_2): second wobble.

Your command passes through all three; what you feel is the output of the chain.

5.2 What co-contraction changes

Co-contraction ξ does **not** change (11). It changes what you **experience** because your arm filters the signal:

$$(\tau_d, \tau_1, \tau_2)_{\text{felt}} = \left(\tau_d, \frac{\tau_1}{\rho(\xi)}, \frac{\tau_2}{\rho(\xi)} \right) \quad (12)$$

At $\xi = 3$, $\rho \approx 34$:

$$\tau_{1,\text{felt}} \approx 0.003 \text{ s}, \quad \tau_{2,\text{felt}} \approx 0.0015 \text{ s}$$

During a 0.8 s reach these are invisible; you feel only $\tau_d = 1$ s.

5.3 Discrete-time simulation

The code advances time with step $\Delta t = 0.01$ s. Each first-order lag with time constant τ uses the difference equation (`cascade_exact`):

$$y[k] = a y[k-1] + (1-a) u[k], \quad a = e^{-\Delta t/\tau} \quad (13)$$

Learner 1 uses an older variant (`cascade`) with one-sample delay per stage; bias analysis uses `cascade_exact` so stiff limiting bias is not masked by discretisation artefact.

5.4 Movement and exploration

Each trial command is:

$$u(t) = u_{\text{ref}}(t) + u_{\text{explore}}(t) \quad (14)$$

- u_{ref} : smooth out-and-back **minimum-jerk reach** (`reach_reference`), duration t_{move} .
- u_{explore} : **band-limited noise** (`band_limited_noise`) — wiggles your arm can actually produce. *Not white noise.*

Lay example: Why band-limited exploration matters

White noise contains all frequencies, so it would excite every mode at once. Then there is no penalty for trying to identify all three lags from trial 1, and the sample-complexity story collapses. Real reaches are smooth; fast modes need **fast movement** to show up.

Felt output with noise:

$$y(t) = \text{cascade}_{\text{exact}}(\text{felt}(\tau, \xi), u) + \text{noise} \quad (15)$$

6 Part IV — Trial failure (surprise)

Thrush: “If stabilisation is not there the ball goes out, the task fails, there is no learning.” The code models this as:

$$\text{surprise} = \max_t |y(t) - \hat{y}(t)| \quad (16)$$

$$\text{tolerance} = S_BASE \frac{\xi}{\xi_{\text{soft}}}, \quad \xi_{\text{soft}} = 0.7 \quad (17)$$

If $\text{surprise} > \text{tolerance}$ the trial **fails** at the first crossing; only data **before** failure are used for learning (if enough samples remain).

Lay example: Failure

Soft arm ($\xi = 0.7$): tolerance ≈ 0.15 . Small prediction errors fail the trial.
 Stiff arm ($\xi = 3$): tolerance ≈ 0.64 . Same wrong model may survive.
 So stiffness buys **safety while wrong**, not magic identification speed.

7 Part V — The bias law (central finding)

7.1 Low-frequency expansion

Expand (11) for small s (slow movements):

$$G(s) = \frac{1}{(1 + \tau_d s)(1 + \tau_1 s)(1 + \tau_2 s)} = 1 - (\tau_d + \tau_1 + \tau_2)s + \mathcal{O}(s^2) \quad (18)$$

A **first-order learner** uses $\hat{G}(s) = 1/(1 + \hat{\tau}s) = 1 - \hat{\tau}s + \dots$. Matching the linear term in s :

$$\boxed{\hat{\tau} = \tau_d + \tau_1 + \tau_2} \quad (19)$$

Bias on the dominant time constant when fitting one lag to a three-lag world:

$$\text{bias} = \hat{\tau}_d - \tau_d \approx \tau_1 + \tau_2 = 0.15 \text{ s} \quad (20)$$

7.2 Effect of co-contraction

Under stiffening, fast lags in the *felt* signal are divided by $\rho(\xi)$:

$$\boxed{\hat{\tau}(\xi) = \tau_d + \frac{\tau_1 + \tau_2}{\rho(\xi)}, \quad \text{bias}(\xi) = \frac{\tau_1 + \tau_2}{\rho(\xi)}} \quad (21)$$

Measured in `bias.py` (12 reaches per ξ , least squares)

ξ	ρ	fitted bias (s)	predicted bias (s)
0.7	1.0	0.180	0.150
1.2	3.5	0.040	0.043
2.0	13.9	0.005	0.011
3.0	34.0	0.001	0.004
4.0	62.0	0.0006	0.0024

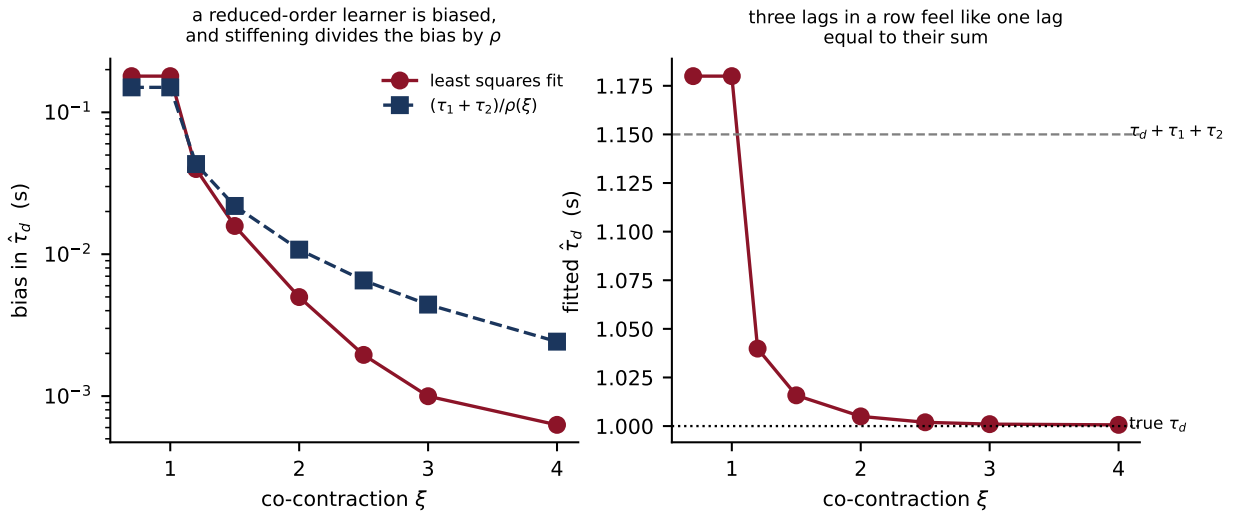


Figure 1: **Bias law.** Left: bias in $\hat{\tau}_d$ vs ξ (data vs $(\tau_1 + \tau_2)/\rho(\xi)$). Right: fitted $\hat{\tau}_d$ approaches true $\tau_d = 1 \text{ s}$ as ξ increases.

Lay example: Bias law

You think the cup has one time constant. Soft arm: you measure $\approx 1.18 \text{ s}$ (sum of all three). Stiff arm ($\xi = 4$): you measure $\approx 1.0006 \text{ s}$, almost the true 1.0 s . Co-contraction did not change the cup — it stopped the fast parts from polluting your **simple** estimate.

8 Part VI — Learner 1 (gradient descent, order unlocked by hand)

8.1 What it believes

Learner 1 (`learner.py`, `experiment.py`) always knows the structure is three lags. It estimates $\hat{\tau}_d, \hat{\tau}_1, \hat{\tau}_2$ but only updates the first n_{unlocked} of them; the rest are fixed at 0 (pass-through).

Initial guesses: $\hat{\tau} = (0.30, 0.03, 0.015)$ s (wrong on purpose).

8.2 Prediction

Given command u and stiffness ξ :

$$\hat{y}(t) = \text{cascade}(\text{felt}(\hat{\tau}, \xi), u) \quad (22)$$

8.3 Learning rule

Keep the last 3 trials. Stack residuals $r = y - \hat{y}$. Cost $\mathcal{C} = \frac{1}{2} \sum r^2$. Parameters stored as $q = \log \tau$. Each trial, take 12 steps of gradient descent:

$$q \leftarrow q - \eta \frac{J^\top r}{m}, \quad J_{tk} = \frac{\partial r_t}{\partial q_k} \quad (23)$$

with learning rate $\eta = 3$.

Plain-language guide: Gradient descent in one sentence

Compute which direction in parameter space would reduce prediction error; take a fixed-size step that way; repeat. Slow when the problem is **ill-conditioned** (some directions barely change the output but look different in parameter space).

8.4 Conditions S1–S5

Code	Tuning schedule	What is imposed
S1	soft, fast, 3 modes from start	deep end
S2	$\xi : 3 \rightarrow 1.3 \rightarrow 0.7$, slow→fast, unlock $1 \rightarrow 2 \rightarrow 3$	Thrish curriculum
S3	stiff throughout, 1 mode only	never learns transients (by rule)
S4	soft, speed staged, unlock $1 \rightarrow 2 \rightarrow 3$	bandwidth channel only
S5	random ξ per trial	control

8.5 Results (20 seeds, 80 trials)

Learner 1 refuted the “curriculum learns faster” claim

	median trials to criterion	reached	final error	failed trials
S1 deep end	47	100%	0.043	10.4
S2 curriculum	80	55%	0.105	0.0
S3 stiff forever	—	0%	0.439	0.0

S2 was **33 trials slower** than S1 ($p = 1.6 \times 10^{-7}$). Curriculum won on **safety** (zero failures), not speed.

Two diagnostic facts (`diagnose.py`):

1. $\kappa(\Phi)$ for the full third-order fit rises from $\sim 5.5 \times 10^3$ (soft) to $\sim 10^{16}$ (stiff): stiffening destroys identifiability of transients in the *full* model.
2. Directions that are hardest to learn also change behaviour least ($28\times$ cheaper in impulse response for $5500\times$ slower curvature): sample-complexity bounds on parameters do not imply behavioural advantage.

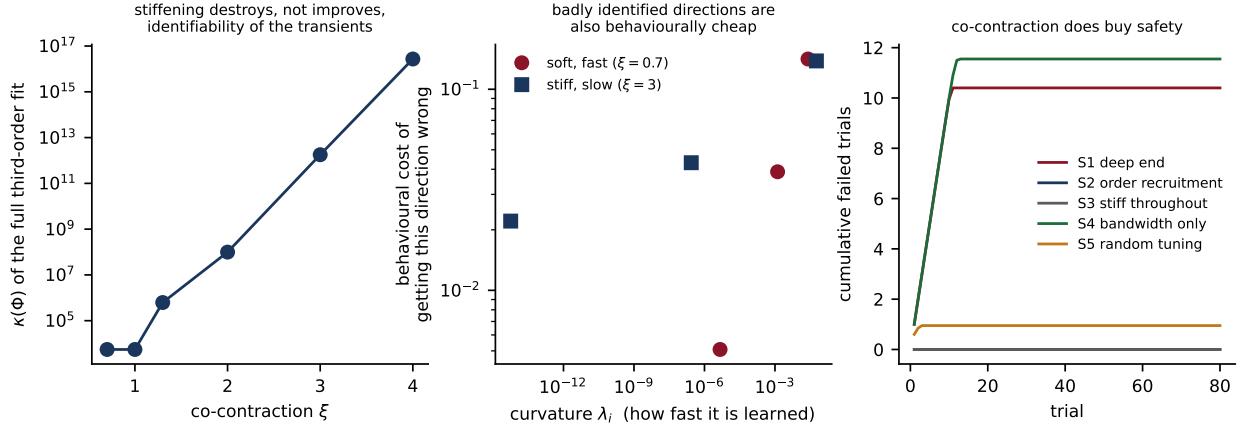


Figure 2: **Learner 1 diagnostics.** Stiffening destroys full-model identifiability; ill-conditioned directions are behaviourally cheap; curriculum reduces failed trials.

9 Part VII — Learner 2 (two-timescale EKF + ARD)

Learner 2 fixes two defects of Learner 1:

1. **Misspecification:** Learner 1 always used three lags when unlocked, so it could not exhibit the bias law (21).
2. **Uncertainty:** Learner 1 hard-locked modes; Learner 2 must show order reduction *emerging* from data, and must not random-walk on unobservable modes.

9.1 State vector

$$q = [\log \hat{\tau}_d, \log \hat{\tau}_1, \log \hat{\tau}_2, w_1, w_2]^\top \quad (24)$$

$w_1, w_2 \in [0, 1.2]$ are **participation weights**: how much of each transient lag is active in the internal model.

9.2 Blend dynamics (why not just set $\tau = 0$?)

Each transient stage applies:

$$\text{blend}(\tau, w)[x] = (1 - w)x + w \cdot \text{lag}(\tau)[x] \quad (25)$$

- $w = 1$: full lag with time constant τ .
- $w = 0$: pass-through (mode “off” but sensitivity to w remains).

Full prediction chain (`kalman_learner._h`):

$$\begin{aligned} x_0 &= \text{lag}(\hat{\tau}_d)[u] \\ x_1 &= \text{blend}(\hat{\tau}_1/\rho(\xi), w_1)[x_0] \\ \hat{y} &= \text{blend}(\hat{\tau}_2/\rho(\xi), w_2)[x_1] \end{aligned} \quad (26)$$

Lay example: Volume knobs vs deleting a mode

Wrong approach: “mode off” = set $\tau_2 = 0$. Then the model’s output does not depend on τ_2 ; the gradient is zero; you can never turn the mode back on when you relax.

Right approach: “mode off” = $w_2 = 0$. Output unchanged, but $\partial \hat{y} / \partial w_2 \neq 0$. When soft data arrive, w_2 can rise again.

9.3 Effective order

$$n_{\text{eff}} = 1 + w_1 + w_2 \quad (27)$$

Roughly: “how many modes are currently paying for themselves in the model?” Not imposed — read from the posterior weights.

9.4 Extended Kalman Filter (EKF) — full plain-language guide

Plain-language guide: Kalman filter

You hold a **belief** (best guess $\hat{q}$) and an **uncertainty** (covariance P). Each trial:

1. **Predict:** allow belief to drift ($P \leftarrow P + Q$).
2. **Update:** compare prediction to data; move belief toward what explains data; shrink uncertainty in directions the data constrain.

If a parameter is not observable this trial, its uncertainty does not shrink — you “know you don’t know.”

Measurement model: $\hat{y} = h(q, u, \xi)$ via (26) (nonlinear).

Linearise around current q :

$$e = y - h(q), \quad H = \left. \frac{\partial h}{\partial q} \right|_q \quad (28)$$

Information-form update (what the code implements each trial, 3 iterated refinements):

$$P_{\text{pred}}^{-1} = P^{-1} + Q^{-1} \quad (29)$$

$$A = P_{\text{pred}}^{-1} + \frac{H^\top H}{R} + \text{diag}(\alpha) \quad (30)$$

$$P_{\text{post}} = A^{-1} \quad (31)$$

$$q \leftarrow q + P_{\text{post}} \left(\frac{H^\top e}{R} - \alpha(q - q_{\text{target}}) - P_{\text{pred}}^{-1}(q - q_{\text{pred}}) \right) \quad (32)$$

Symbols:

- Q — process noise: how much you allow beliefs to drift between trials (small on $\log \hat{\tau}_d$, larger on transients and weights).
- R — measurement variance; inflated to $\max(R_{\text{std}}^2, \text{mean}(e^2))$ early so the filter is not overconfident while the model is still wrong.
- α — ARD penalties (next section).

Lay example: EKF

Day 1: you guess cup time constant 0.3s, very unsure (P large). You reach, feel sluggish response. Filter moves estimate toward 0.8s and reduces uncertainty on $\log \hat{\tau}_d$. Fast modes stiffened away: columns of H for w_1, w_2 near zero $\Rightarrow$ those uncertainties barely shrink $\Rightarrow$ you do not pretend to have learned them.

9.5 ARD — Automatic Relevance Determination

Plain-language guide: ARD in plain language

Each participation weight w_i has a penalty α_i pulling it toward 0 (“this wobble probably does not matter”). After each update:

$$\gamma_i = 1 - \alpha_i P_{ii}, \quad \alpha_i \leftarrow \frac{\gamma_i}{\max(w_i^2, \varepsilon)} \quad (33)$$

If the data do not support mode i , P_{ii} stays large, α_i stays moderate, and w_i is shrunk. If soft fast data strongly need mode i , w_i grows and stays.

This is how n_{eff} drops toward 1 while stiff and rises after relaxation without anyone typing “unlock mode 2 now.”

9.6 Two timescales

Process noise diagonal (rough magnitudes):

$$Q = \text{diag}(10^{-5}, 2 \times 10^{-3}, 2 \times 10^{-3}, 2 \times 10^{-3}, 2 \times 10^{-3})$$

Dominant $\log \hat{\tau}_d$ drifts slowly (stable memory). Weights and fast τ 's drift faster (forgetful channel).

9.7 Conditions K1–K5

Same schedules as S1–S5 except:

- **No hand unlock** — all three τ 's and both w 's always exist.
- K2/K4 advance stiffness/speed when $\text{sd}(\log \hat{\tau}_d) < 0.04$ (confident on dominant mode).

9.8 Results (20 seeds, 60 trials)

Learner 2 confirms mechanism; does not confirm speed claim

	trials	final error	peak $ \hat{\tau}_d - \tau_d $	n_{eff}	failed
K1 deep end	5	0.031	0.691 s	2.18	4.0
K2 confidence-gated	19	0.030	0.087 s	2.18	0.0
K3 stiff forever	—	0.404	0.013 s	1.00	0.0
K4 bandwidth only	4	0.030	0.026 s	2.19	1.1
K5 random	35	0.076	0.040 s	1.92	0.7

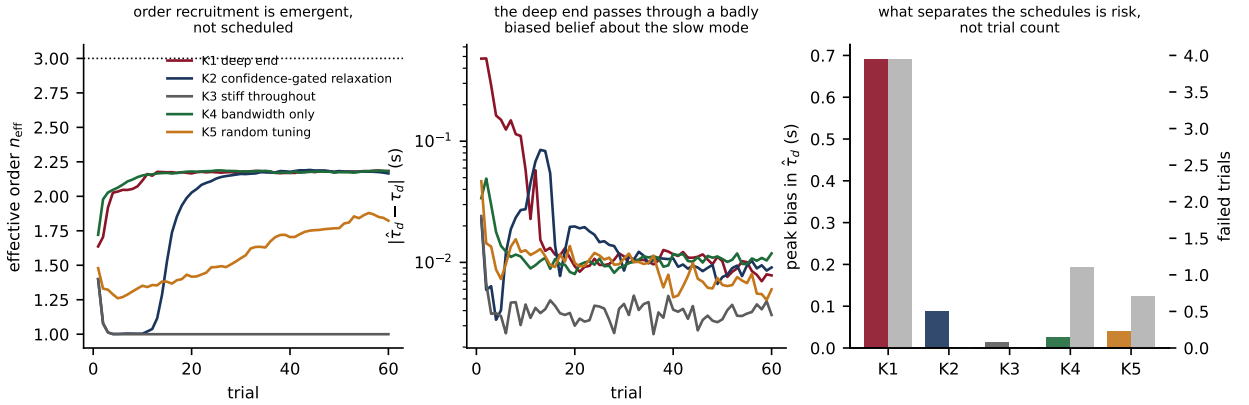


Figure 3: **Learner 2**. Left: n_{eff} over trials (K2 holds at 1.0 while stiff, then recruits). Centre: peak bias in $\hat{\tau}_d$. Right: risk vs peak bias across schedules.

Lay example: Reading K2 vs K1

Both end with similar final error (~ 0.03) and $n_{\text{eff}} \approx 2.2$. K1 gets there in ~ 5 trials but passes through $\hat{\tau}_d$ wrong by up to ~ 0.7 s and fails ~ 4 trials. K2 takes ~ 19 trials but peak bias is ~ 0.09 s and **zero failures**. The curriculum is a **safer path**, not a shorter one.

10 Part VIII — Scoring metrics (every formula)

Relative impulse error (how wrong is the learned object model?):

$$\text{err} = \frac{\|y_{\text{imp}} - \hat{y}_{\text{imp}}\|_2}{\|y_{\text{imp}}\|_2} \quad (34)$$

Trials to criterion: first trial where $\text{err} < 0.10$ for 3 consecutive trials.

Condition number (Learner 1 diagnostics):

$$\kappa(\Phi) = \frac{\lambda_{\max}(H^\top H)}{\lambda_{\min}(H^\top H)} \quad (35)$$

Large κ : some parameter directions are nearly invisible in the data.

11 Part IX — Conclusions and discussion

11.1 What we conclude

1. **Co-contraction manufactures time-scale separation** through $\rho(\xi)$ (9), matching the mass-spring-damper story (1)–(10).
2. **The precise benefit for a low-order learner is bias reduction** (21), not generic “fewer parameters = fewer trials.” Stiffening divides misspecification bias by ρ . This is measurable in people.
3. **Progressive order recruitment can emerge** (Learner 2 K2): n_{eff} holds near 1 while stiff, rises when co-contraction falls and data expose transients.
4. **Staying stiff forever leaves you at first order** (K3): $n_{\text{eff}} = 1$, unbiased $\hat{\tau}_d$, plateau error ≈ 0.40 — discovered, not coded.
5. **Curriculum reduces risk, not trial count** in these simulations: K2/S2 have zero failed trials; deep-end schedules fail more while wrong.
6. **Humans starting stiff is rational** if the objective is “do not lose trials to catastrophic failure while building a coarse model,” not “minimise trial count on a correctly specified full-order estimator.”

11.2 What we do *not* conclude

Do not claim these in the proposal

- “S2/K2 reaches criterion in fewer trials than S1/K1” — **false** in both learners for the reported metrics.
- “Sample complexity inequality implies faster behavioural learning” — refuted for Learner 1 (ill-conditioned directions are behaviourally cheap).
- “Bandwidth staging alone (K4) is worse than full curriculum” — K4 is among the fastest schedules in Learner 2.
- “Learners always recruit all three modes” — $n_{\text{eff}} \approx 2.2$ because the third mode often does not pay for itself under band-limited excitation.

11.3 Recommended claim for the URF four-pager

A learner that treats the coupled system as lower order than it truly is inherits a bias set by the neglected transients. Co-contraction divides that bias by the separation ratio $\rho(\xi)$, yielding an unbiased estimate of the dominant mode while temporarily suppressing effective order. A uncertainty-aware learner exhibits progressive recruitment when co-contraction relaxes, reaching the same asymptotic model as an unstiffened learner but with substantially lower peak error in the dominant mode and fewer failed trials.

11.4 Open next steps (not yet simulated)

- Close the loop with a certainty-equivalent controller (instability during learning is part of the phenomenon).
- Sweep cost of failed trials (recovery delay).
- Test $\hat{\tau}_d$ bias vs EMG-derived ξ in WP1 participants.

12 Appendix — Code map

File	Role
<code>order_reduction/plant.py</code>	Object, $\rho(\xi)$, cascades, reach, exploration
<code>order_reduction/learner.py</code>	Learner 1: gradient descent
<code>order_reduction/experiment.py</code>	S1–S5 trial loop
<code>order_reduction/diagnose.py</code>	$\kappa(\Phi)$ and behavioural cost
<code>order_reduction/bias.py</code>	Bias law verification
<code>order_reduction/kalman_learner.py</code>	Learner 2: EKF + ARD
<code>order_reduction/experiment2.py</code>	K1–K5 trial loop
<code>scripts/run_curriculum.sh</code>	Run Learner 1 figures
<code>scripts/run_learner2.sh</code>	Run Learner 2 figures